

สอนมะขาม - ฉบับละเอียด

AI ในตลาด เลือกยังไงดี?

ทำไม Constistant ยังอ่านคานไม่ออก โมเดลใหญ่ช่วยได้จริงไหม แล้วถ้า
อยากได้โมเดลที่ไม่ใช่ Qwen เหลือตัวไหนให้เลือกบ้าง — เล่าทั้งเรื่อง
ตั้งแต่ต้นจนจบ พร้อมหลักฐานทุกจุด

วันที่ 26 ส.ค. 2569 สำหรับ โปรเจกต์ Constistant งบ การ์ดจอเช่า ≤ \$4/ชม.

สารบัญ

- โมเดลคืออะไร มองข้างในคืออะไร — อธิบายแบบเด็ก 5 ขวบ
- ทำไมคานถึงอ่านไม่ออก — สืบจนเจอสาเหตุจริง (ไม่ใช่ที่คิดไว้แต่แรก)
- ไล่หาโมเดล 100-250B ที่ไม่ใช่ Qwen — ทุกตัวที่เช็ค พร้อมเหตุผลตัดออก
- เอาไปต่อกับ Constistant ได้ยังไง
- ราคาการ์ดจอเช่าจริง วันนี้ (26 ส.ค. 2569)
- สิ่งที่ควรทำต่อไป เรียงลำดับ

ส่วนที่ 1

โมเดลคืออะไร มองข้างในคืออะไร

อธิบายแบบให้เด็ก 5 ขวบก็เข้าใจ ไม่มีศัพท์เทคนิคเลยสักคำ — อ่านส่วนนี้จบแล้ว
จะเข้าใจทุกอย่างที่พูดในส่วนถัดไปได้หมด



โมเดล = คนทำอาหารคนหนึ่ง

ลองนึกภาพเชฟคนหนึ่ง เราส่งรูปแบบก่อสร้างให้เขาดู แล้วขอให้เขาเขียนสรุปออกมา
เป็นรายการ (เหมือนใบสั่งของ) — "มีคาน B1 กี่ตัว ยาวเท่าไร เหล็กข้างในกี่เส้น" เชฟ
แต่ละคนฝีมือไม่เท่ากัน บางคนอ่านตัวเลขเล็กๆ ได้ไว บางคนจำสูตรได้เยอะ บางคนขยัน
จดละเอียดกว่าคนอื่น "โมเดล" ก็คือเชฟคนนั้น — Constistant ตอนนี้จ้างเชฟชื่อ
Qwen3.6-35B-A3B อยู่



เชฟมี "ตา" กับ "สมอง" คนละส่วนกัน

ตอนเชฟทำงาน เขาใช้ตาดูภาพก่อน (ส่วนนี้เรียกว่า "vision encoder") แล้วค่อยเอาสิ่งที่
เห็นไปคิดต่อในหัว (ส่วนนี้คือ "สมอง" หรือ LLM) สองส่วนนี้บางทีสร้างมาจากคนละที่
กันเลย แล้วเอามาต่อกันทีหลัง เหมือนเอาแว่นตาชุมพิเศษ มาสวมให้คนคิดเลขเก่งคน
หนึ่ง



ตา

มองภาพแบบก่อสร้าง



สมอง (LLM)

คิด จำ เขียนคำตอบ

โมเดล VLM ตัวหนึ่ง = "ตา" (มองภาพ) + "สมอง" (คิด/เขียน) สองส่วนนี้อาจสร้างมา
คนละที่กัน แล้วเอามาต่อกันทีหลัง



"สมองข้างใน" = สูตรที่เชฟคนนั้นไปเรียนมา

เชฟทุกคนต้องไปเรียนทำอาหารจากที่ไหนสักแห่งก่อนจะมาเปิดร้าน บางคนไปเรียนกับ
ครูคนเดียว (สูตรเดียวกันเป๊ะ) แต่ใส่ชุดเชฟคนละยี่ห้อ หน้าตาเป็นคนละร้าน แต่ถ้า
แกะดูจริงๆ ฝีมือทำอาหารมาจากครูคนเดิม — วงการ AI ก็เป็นแบบนี้เหมือนกัน บาง
บริษัทไม่ได้สอนสูตรเอง แต่ "ยืม" สมองของบริษัทอื่นมาใส่ในตัวใหม่ แค่เปลี่ยนชื่อ
เปลี่ยนหน้าตาข้างนอก — เดี่ยวในส่วนที่ 3 จะเห็นว่านี่คือสิ่งที่เกิดขึ้นจริงกับหลายโมเดล
ที่เราไปเช็คมา



"B" (บิลเลียน) = ขนาดสมอง ไม่ใช่ความฉลาดเสมอไป

ทุกครั้งที่เห็นเลข B (เช่น 35B, 122B) คือขนาดของสมอง ยิ่งเลขเยอะ สมองยิ่งใหญ่ ยิ่งจำได้เยอะ — แต่สมองใหญ่ไม่ได้แปลว่าอ่านตัวหนังสือเล็กๆ ได้ดีกว่าเสมอ เหมือนคนตัวใหญ่ไม่ได้แปลว่าสายตาดีกว่าคนตัวเล็ก บางทีสมองเล็กที่ฝึกมาเฉพาะทางกลับอ่านได้แม่นยำกว่าสมองใหญ่ที่ฝึกมาแบบกว้างๆ

หลักฐานจริงที่พบระหว่างค้นข้อมูล

InternVL3 รุ่นเล็ก 8B ได้คะแนนอ่านตัวหนังสือ (OCRBench) 881 ส่วนรุ่นใหญ่ 78B ได้แค่ 847 — ตัวเล็กกว่าชนะตัวใหญ่กว่าตรงๆ และตัวจิ๋ว 1.2B ชื่อ MinerU2.5 ยังชนะตัวยักษ์ 72B บนสนามแข่งอ่านเอกสาร (90.67 ต่อ 87.02) — เล็กกว่ากัน 60 เท่า แต่ชนะ เพราะงานอ่านตัวหนังสือเล็กๆ ขึ้นกับวิธีที่ "ตา" มองภาพมากกว่าขนาดสมอง



MoE คืออะไร ("ทั้งหมด" กับ "คิดจริง")

บางโมเดลเขียนว่า "106B ทั้งหมด / 12B คิดจริง" — เหมือนร้านอาหารที่มีเชฟเก่งเฉพาะทางอยู่หลายคน (เชฟผัด เชฟทอด เชฟซูป) แต่ทำอาหารจานหนึ่ง ไม่ต้องเรียกเชฟมาทั้งร้าน เรียกมาแค่ 2-3 คนที่ถนัดจานนั้น — โมเดลแบบนี้เรียกว่า "MoE" (มีผู้เชี่ยวชาญหลายคน แต่ทำงานจริงแค่บางคนต่อครั้ง) ข้อดีคือเร็วกว่าเรียกทุกคนมาทำงาน แต่ตอนฝึกสอนต้องสอนผู้เชี่ยวชาญทุกคนแยกกัน ซึ่งกินพื้นที่ความจำเยอะกว่าที่คิด — เดียวจะเห็นตัวเลขจริงในส่วนที่ 5

สรุปสั้นๆ ก่อนไปต่อ: เลือกโมเดล = เลือกเชฟ ต้องดู 3 อย่าง — (1) ฝีมือจริงกับงานที่จะให้ทำ ไม่ใช่แค่ดูป้ายขนาด B (2) เรียนสูตรมาจากใคร สมองข้างในใช้ของเจ้าอื่นซ่อนอยู่ไหม (3) จ้างได้ในราคาที่จ่ายไหว

ส่วนที่ 2

ทำไมคนถึงอ่านไม่ออก

ก่อนจะไปหาโมเดลใหม่ เราต้องรู้ก่อนว่า "ปัญหาจริง" คืออะไร เพราะถ้าเราผิด เปลี่ยนโมเดลไปก็ไม่ช่วยอะไร — เรื่องนี้มีการกลับความคิดกลางทางด้วย น่าสนใจ ตรงนี้

ทฤษฎีแรก: ภาพถูกบีบเล็กจนอ่านไม่ออก

Constistant ตอนนี้ (ตัว t03) วัดคะแนน "อ่านข้อความถูกไหม" ได้ผลแบบนี้:

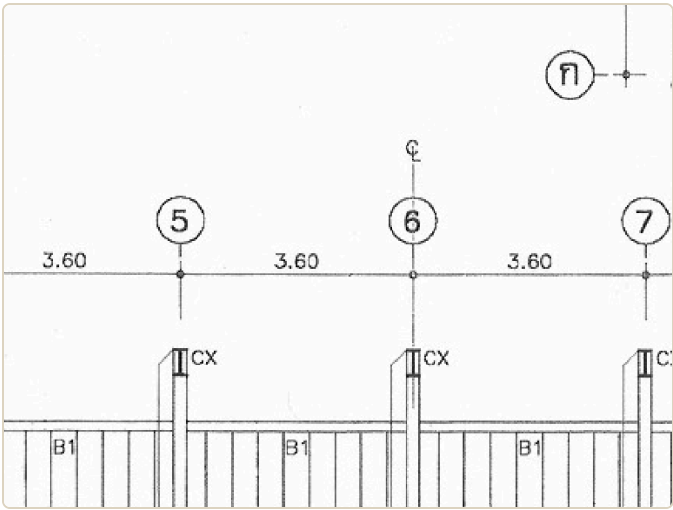
งาน	เพดานสูงสุดที่ได้	คะแนนจริงที่ได้	อ่านว่า
อ่านข้อความ (plan_beam)	88%	0%	❌ ผิดปกติมาก
อ่านชื่อพื้น (plan_slab)	92%	20%	❌ ยังต่ำ
อ่านชื่อฐานราก (plan_footing)	75%	33%	🟡 พอไหว
อ่านตาราง (schedule)	61%	22%	🟡 พอไหว

ตัวเลข "เพดานสูงสุด" หมายถึงต่อให้อ่านถูกทุกจุดจริงๆ คะแนนสูงสุดที่เป็นไปได้คือเท่านี้ (บางชื่อในข้อมูลจริงเป็นชื่อที่คนจัดเองแบบบรรยาย ไม่มีพิมพ์อยู่บนแบบเลย โมเดลจะเดาถูกไม่ได้อยู่แล้ว) — งาน "อ่านข้อความ" ทำได้สูงสุดถึง 88% แต่ได้จริงแค่ 0% ตัวเลขนี้ผิดปกติมากจนต้องสืบต่อ

ตอนแรกสงสัยว่าเพราะภาพแบบก่อสร้างเป็นแผ่นใหญ่มาก (ขนาดกระดาษ A1) พอส่งให้โมเดลดู ระบบจะบีบภาพให้เล็กลงก่อนเพื่อประหยัดพื้นที่ประมวลผล — ตัวหนังสือขนาด 2 มิลลิเมตรบนแบบ พอถูกบีบแล้วอาจเหลือแค่ไม่กี่พิกเซล อ่านไม่ออกแน่ ยิ่งภาพที่ใหญ่สุดในชุดข้อมูล (4631×3473 พิกเซล) ยิ่งโดนบีบหนัก — เหลือแค่ 61% ของความละเอียดเดิม

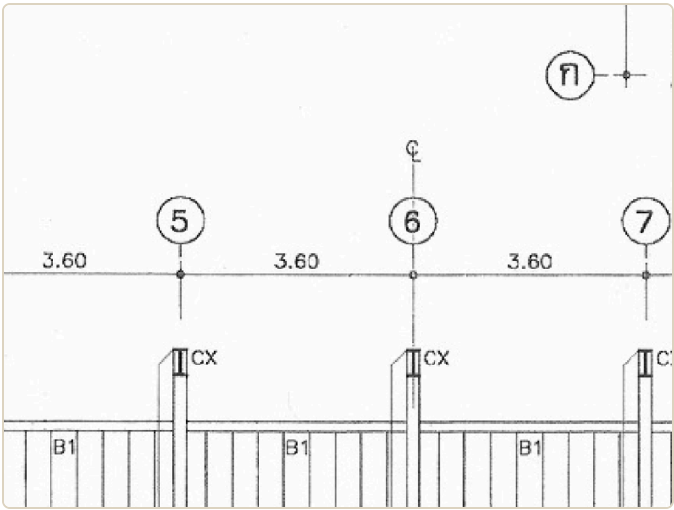
พิสูจน์ด้วยตา — ตัดภาพจริงมาเทียบ

แทนที่จะเชื่อทฤษฎีตัวเลขอย่างเดียว เราตัด (crop) ภาพแบบก่อสร้างจริงจากชุดข้อมูล (บ้านใหญ่ 1 ชั้น หน้า 15 — แผ่นที่ใหญ่สุดและโดนบีบหนักสุดในทั้งชุด) แล้วเทียบสองแบบ: ต้นฉบับเทียบกับสิ่งที่โมเดลเห็นจริงหลังถูกบีบ



ต้นฉบับ (ไม่บีบ)

ตัด 1200×900 พิกเซลจากกลางแผ่น



สิ่งที่โมเดลเห็นจริง

บีบเหลือ 70.3% แล้วขยายกลับมาให้ดูง่าย

ผลที่เห็นด้วยตา

ทั้งสองภาพอ่านออกเหมือนกันทุกตัวอักษร — ชื่อคาน "B1" ป้ายเสา "CX" ตัวเลขระยะ "3.60" และหมายเลขกริด ①②③ ยังคมชัดพอกันทั้งคู่ นี่คือหลักฐานที่พลิกทฤษฎีแรก: การบีบภาพไม่ใช่สาเหตุที่คะแนนออกมา 0%

สืบต่อ — เจอสาเหตุจริง: คำตอบยาวเกินไป

เมื่อตัดทฤษฎีเรื่องภาพออกไปแล้ว กลับไปดูสิ่งที่โมเดลต้อง "เขียนออกมา" เป็นคำตอบ — งานอ่านคาน (plan_beam) ต้องให้โมเดลเขียนรายการคานทั้งหมดบนแผ่นออกมาเป็นก้อนข้อความยาวๆ ก่อนเดียว วัดความยาวจริงของคำตอบที่ถูกต้อง (ground truth) ในแต่ละงานแล้วเจอแพทเทิร์นชัดเจนมาก:

งาน	คำตอบยาวสุด (ตัวอักษร)	≈ ความยาวเป็นหน่วยคิด	คะแนนจริง
อ่านชื่อคาน	20,224	≈ 9,192 หน่วย	0%
อ่านหมายเลขกริด	19,114	≈ 8,688 หน่วย	(ยังไม่วัด)
อ่านชื่อพื้น	14,417	≈ 6,553 หน่วย	20%
อ่านฐานราก	8,871	≈ 4,032 หน่วย	33%
อ่านตาราง	9,453	≈ 4,296 หน่วย	22%

งานที่ต้องเขียนคำตอบยาวที่สุด คือ "อ่านชื่อคน" งานเดียวกันที่ได้คะแนน 0% — และงาน "อ่านฐานราก" ที่อ่านจากแผ่นเดียวกัน ความละเอียดภาพเดียวกัน ขนาดตัวหนังสือเดียวกัน แต่คำตอบสั้นกว่า 2.3 เท่า กลับได้คะแนนสูงกว่ามาก (33% เทียบกับ 0%)

สรุปสาเหตุที่แท้จริง

โมเดลไม่ได้ "มองไม่เห็นคน" — มันเห็นครบ อ่านออกครบ แต่เขียนรายการคนทั้ง 45 ตัวให้ถูกต้องครบทุกตัวในคำตอบเดียวไม่ได้ พลาดจุดใดจุดหนึ่งกลางทาง ทำให้ไฟล์ JSON ทั้งก้อนเสียหาย นับเป็นคะแนน 0% ทั้งที่เขียนถูกไปแล้ว 44 ใน 45 ตัวก็ตาม — เหมือนให้เชฟทำสูตรอาหาร 45 ขั้นตอนโดยห้ามพลาดผิดสักคำ พลาดคำเดียวกลางทางถือว่าทั้งหมดผิด ทั้งที่จำได้เกือบหมดจริงๆ

ทางแก้ที่แนะนำ (ยังไม่ได้ลงมือทำ): แบ่งงาน "อ่านคน" เป็นชิ้นเล็กๆ ตามโซนกริดของแบบ (เช่น อ่านทีละช่วงกริด ⑤-⑥ แทนที่จะอ่านทั้งแผ่นทีเดียว) แล้วเอาโค้ดมาต่อคำตอบเข้าด้วยกันทีหลัง — ลดความยาวคำตอบต่อครั้งลง ไม่ใช้การซื้อความละเอียดภาพเพิ่ม หรือเปลี่ยนไปใช้โมเดลที่ใหญ่กว่า

ส่วนที่ 3

ไล่หาโมเดล 100–250B ที่ไม่ใช่ Qwen

แม้จะรู้แล้วว่าปัญหาจริงไม่ใช่ขนาดโมเดล มะขามยังอยากรู้คำตอบของโจทย์นี้ให้ครบ: ขนาดสมอง 100–250 พันล้านพารามิเตอร์ ทุนเองได้จริง ใช้ทำมาหากินได้ และ "สมองข้างใน" ต้องไม่ใช่ของ Qwen ที่ยืมมา — เช็คมามาทั้งหมด 8 ตัว นี่คือทุกตัวพร้อมเหตุผลที่ตัดออกทีละตัว

ตัวที่ตัดออกเพราะสมองยืมมาจาก Qwen

InternVL3-78B

สมองยืมมา

หน้าตาเป็นของ OpenGVLab แต่แกะไฟล์ตั้งค่าดูจริงๆ **สมองข้างในคือ Qwen2.5-72B-Instruct** — เหมือนร้านอาหารเปลี่ยนป้าย แต่เซฟข้างในคนเดิม

น้ำหนักไฟล์	ลิขสิทธิ์	ทุนผ่าน UNSLOTH ได้ไหม
156.8 GB	ใช้ได้ถ้าผู้ใช้ <100 ล้าน/เดือน	ไม่ได้ (มีบั๊กจริง)

หลักฐาน เช็คไฟล์ config.json ตรงๆ พบ llm_config._name_or_path = Qwen2.5-72B-Instruct

InternVL3.5-241B-A28B สมองยืมมา

รุ่นใหม่กว่า แต่แกะไฟล์ตั้งค่าเจอชื่อ Qwen3-235B-A22B เขียนอยู่ในนั้นเลย — สมองคือ Qwen3 ทั้งก่อน แคะห่อด้วยชื่อใหม่ ขนาดก็เกินโจทย์ (100-200B) ไปมากด้วย

น้ำหนักไฟล์	ลิขสิทธิ์	อยู่ในช่วงที่ขอไหม
481 GB	Apache-2.0 (มรดกจาก Qwen3)	เกินไปมาก (241B)

หลักฐาน เช็ค config.json ตรงๆ เจอ llm_config.architectures = Qwen3MoeForCausalLM และ _name_or_path = /checkpoints/Qwen3-235B-A22B

ตัวที่ตัดออกเพราะห้ามใช้เชิงพาณิชย์

Pixtral-Large ห้ามทำมาหากิน

สมองเป็นของ Mistral เองจริง ไม่ยืมใคร แต่ใบอนุญาตเขียนไว้ชัดว่าเป็น Mistral Research License — ใช้ได้แค่เพื่อการวิจัย ห้ามใช้หารายได้เด็ดขาด ถ้าจะใช้เชิงพาณิชย์ต้องจ่ายเงินซื้อสัญญาแยกกับ Mistral

น้ำหนักไฟล์	ลิขสิทธิ์	CONSTANT ใช้ได้ไหม
248.3 GB	ห้ามพาณิชย์เด็ดขาด	ไม่ได้ ผิดสัญญา

NVLM-D-72B ห้ามทำมาหากิน + ทุนไม่ได้

ของ NVIDIA เอง สมองไม่ได้ยืมใคร (มี Qwen2-72B ผสมอยู่บ้างแต่ทีมพัฒนาเองทั้งหมด) แต่ติดใบอนุญาต CC-BY-NC-4.0 (ห้ามใช้เชิงพาณิชย์) ทันที้ และยังไม่มีเครื่องมือทุนตัวไหนรองรับเลย แม้แต่ vLLM (ตัวรันโมเดล) ก็ยังรันไม่ได้ดี NVIDIA เองก็ยอมรับว่ายังแก้ไขไม่เสร็จ

น้ำหนักไฟล์

ลิขสิทธิ์

ทุนได้ไหม

176.9 GB

ห้ามพาณิชย์ (CC-BY-NC-4.0)

ไม่มีเครื่องมือรองรับเลย

ตัวที่ตัดออกเพราะทุนไม่ได้จริง

Molmo-72B

ทุนไม่ได้จริง

ของ AllenAI สมองเป็นของตัวเองจริง แต่ไม่มีเครื่องมือทุนมาตรฐานตัวไหนรองรับเลย AllenAI เองก็บอกตรงๆ ในกระทู้ถามตอบว่ายังไม่มีสคริปต์ทุนให้ ต้องเขียนเองทั้งหมด และมีคนในชุมชนรายงานว่าวิธีทุนแบบประหยัด (LoRA) ใช้กับโมเดลนี้แล้วไม่เสถียร

น้ำหนักไฟล์จริง

ลิขสิทธิ์

ทุนได้ไหม

~147 GB (bf16)

ใช้ได้ (Apache-2.0)

ไม่มีคู่มือทางการ

หมายเหตุ ตอนแรกเห็นไฟล์ในเว็บโชว์น้ำหนัก 293 GB นาดกใจมาก ไปตรวจดูจริงพบว่าเก็บเป็นไฟล์ตัวเลขละเอียด 32 บิต (แทนที่จะเป็นแบบมาตรฐาน 16 บิตที่ทุกเจ้าใช้กัน) พอแปลงกลับมาความจริงคือ ~147 GB — เป็นบทเรียนว่าต้องเช็คแบบเลขให้ติก่อนเชื่อตัวเลขที่โชว์ในเว็บ

ตัวที่รอดถึงรอบสุดท้าย

เหลือ 2 ตัวที่ผ่านทั้งสามด่าน (สมองไม่ยืมมา + ใช้พาณิชย์ได้ + ทุนได้จริง):

GLM-4.6V

106B ทั้งหมด / คิดจริง 12B ต่อครั้ง

ของ Zhipu (จีน) สมองเป็นของตัวเองแท้ๆ ไม่ยืมใคร ใบอนุญาตดีที่สุดในกลุ่มทั้งหมดที่เช็คมา (MIT — ใช้ได้ทุกอย่างไม่มีเงื่อนไขเลย) แต่ เครื่องมือทุนยอดนิยม (LLaMA-Factory) มีบั๊กจริงที่ยังไม่มีใครแก้ — คนที่ลงทุนโมเดลตระกูลเดียวกันเจอ error ตั้งแต่ก้าวแรกของการตั้งค่า

น้ำหนักไฟล์	ลิขสิทธิ์	ทุนได้ไหม	พูดไทยเก่งไหม
215.4 GB	MIT — ใช้ได้หมด	มีบ้างจริง ⚠️	ไม่มีข้อมูลเลย

หลักฐาน GitHub issue #9635: error NotImplementedError: get_input_embeddings เมื่อพยายามทุนแบบ LoRA — สถานะยังเปิดค้างไว้ ไม่มีคนแก้

Llama-4-Scout

109B ทั้งหมด / คิดจริง 17B ต่อครั้ง

ของ Meta สมองเป็นของตัวเองแท้ๆ ทุนได้จริงและมีคนทำสำเร็จมาแล้ว (ใช้เครื่องการ์ดจอ 8 ใบ ทุนจบใน 21 นาที) แต่ ทุนได้แค่แบบ "4-bit" (บีบข้อมูลให้เล็กลงก่อนเรียนรู้ เพื่อประหยัดความจำการ์ดจอ) ไม่ใช่แบบเต็มความละเอียดที่ Constistant ใช้ยุดอนนี้ — คุณภาพผลลัพธ์อาจต่างจากที่คาด

น้ำหนักไฟล์	ลิขสิทธิ์	ทุนได้ไหม	พูดไทยเก่งไหม
217.3 GB	ใช้ได้ถ้าผู้ใช้ <700 ล้าน/เดือน	ได้จริง ✅ (แบบประหยัดเท่านั้น)	ไม่มีข้อมูลเลย

หลักฐาน บทความความจริงจาก Fixstars — ทุนสำเร็จบนเครื่อง 8×H100 จบ 3 รอบการเรียนรู้ใน ~21 นาที

คำตอบรวม

ยังไม่มีตัวไหน "ชนะขาด" — และ ปัญหาจริงไม่ได้อยู่ที่ขนาดโมเดล

จากทุกตัวที่มีในตลาดวันนี้ ขนาด 100–250B ที่สมองไม่ใช่ Qwen เลยสักส่วน + ทุนได้จริง + ใช้หากินได้ เหลือรอดแค่ GLM-4.6V กับ Llama-4-Scout — แต่ทั้งคู่มีจุดอ่อนคนละแบบ และไม่มีใครเคยพิสูจน์ว่าอ่านภาษาไทยเก่งแค่ไหน ที่สำคัญกว่านั้น: ปัญหาจริงของ Constistant ตอนนี้ (คานอ่านไม่ออก) พิสูจน์แล้วว่าไม่ได้เกิดจากขนาด

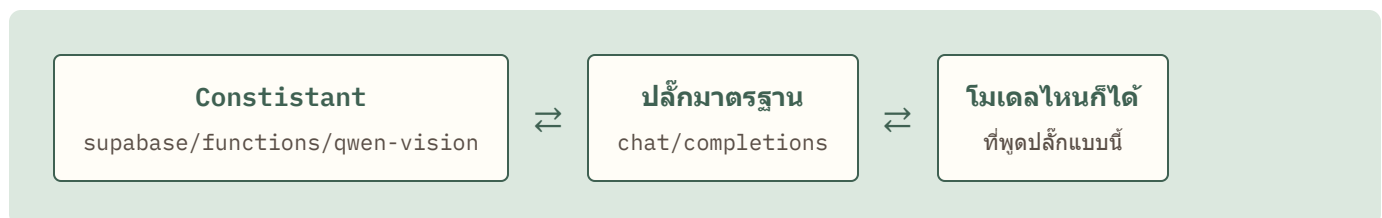
โมเดลหรือความละเอียดภาพ — เปลี่ยนไปโมเดลใหญ่กว่านี้อาจไม่ช่วยอะไรเลย
จนกว่าจะแก้เรื่องความยาวคำตอบก่อน

ส่วนที่ 4

เอาไปต่อกับ Constistant ได้ไหม

ข่าวดี: ต่อกับโค้ดที่มีอยู่แล้วได้ ไม่ต้องแก้อะไรใน Constistant เลย

ตอนนี้ Constistantคุยกับ AI ผ่านช่องทางมาตรฐานตัวหนึ่งที่เรียกว่า chat/completions (เหมือนปลั๊กไฟบ้านที่ใช้กับเครื่องใช้ไฟฟ้าอะไรก็ได้ที่มีปลั๊กแบบเดียวกัน) — โมเดลไหนก็ตามที่เปิดให้เรียกใช้ผ่านช่องทางนี้ เสียบเข้าไปแทนของเดิมได้เลย



ทั้ง GLM-4.6V และ Llama-4-Scout เมื่อทุนเสร็จแล้วเอาไปรันด้วยเครื่องมือมาตรฐาน (ชื่อ vLLM) จะพูดปลั๊กแบบเดียวกันนี้เองอัตโนมัติ — แปลว่าแค่เปลี่ยนที่อยู่เซิร์ฟเวอร์กับชื่อโมเดลในไฟล์เดียว งานสอนโมเดลใหม่กับงานต่อเข้า Constistant คือคนละงานกัน แยกจากกันเลย ไม่ต้องรอให้เขียนโค้ด Constistant เพิ่ม

ทำไมถึงมั่นใจว่าเสียบได้ทันที

ไปเปิดไฟล์จริงในโค้ด Constistant (supabase/functions/qwen-vision/index.ts) เช็ครูปร่าง พบว่ามันเรียก

`{'{'qwenApiHost{'{'}/chat/completions` พร้อมตั้งค่า

`response_format: json_object` — นี่คือรูปแบบมาตรฐานที่โมเดลเปิด (open-

weight) เกือบทุกตัวในตลาดวันนี้รองรับเมื่อรันผ่าน vLLM ไม่ใช่ฟีเจอร์เฉพาะของ Qwen

ส่วนที่ 5

ราคาการ์ดจอเช่าจริง วันนี้

เช็คสดผ่าน vast.ai (ไม่ใช่หน้าเว็บโฆษณา — ยิงคำสั่งดึงข้อมูลตรงจากระบบจริง) เมื่อวันที่ 26 ส.ค. 2569

เดือนก่อน: ตัวเลขที่เคยเข้าใจผิด

มีตัวเลข "\$0.67/ชั่วโมง" สำหรับการ์ดที่ใช้อยู่ตอนนี้ ลอยอยู่ในหน้าเว็บโฆษณาบางที่ — เช็คสดผ่านระบบจริงแล้วไม่มีการลดราคานี้ขายอยู่เลย ถูกสุดจริงคือ \$0.975-1.01/ชั่วโมง ตัวเลข \$0.67 น่าจะเป็นราคาประมูลแบบเสี่ยงโดนแย่ง (ไม่ใช่ราคาจ้างตรง) คนละคอลัมน์กันเลย — เป็นกับดักเดียวกับที่เคยอ่านราคาที่เก็บพื้นที่ผิดเป็นราคาการ์ดจอมาก่อน

การ์ดจอ	ความจำรวม	ราคา/ชั่วโมง	ใช้ได้กับอะไร
RTX PRO 6000 ×1 (ที่ใช้อยู่)	96 GB	\$0.975–1.11	Qwen3.6-35B-A3B ตัวปัจจุบัน
RTX PRO 6000 ×2	192 GB	\$2.00–2.19	InternVL3-78B (ดังมาก)
H200 ×1	141 GB	\$3.29	โมเดลไม่ย่อภาพเลย
8×RTX 4090 (ไม่มี NVLink)	384 GB	\$2.99	เสี่ยง — MoE ขาดถ้าไม่มีสายเชื่อมต่อเร็ว
GLM-4.6V (ต้องการจริง)	~270 GB	ไม่มีการลดเดียวพอ	ต้องหลายใบ
Llama-4-Scout (ต้องการจริง)	~272 GB	ไม่มีการลดเดียวพอ	ต้องหลายใบ

ราคาการ์ดจอเช่าเป็นตลาดเปิด เปลี่ยนได้ทุกชั่วโมงตามคนแย่งกันเช่า — ตัวเลขนี้คือภาพ ณ วันที่เช็ค ต้องเช็คซ้ำก่อนกดเช่าจริงเสมอ

ส่วนที่ 6

สิ่งที่ควรทำต่อไป เรียงลำดับ

1 แบงงาน "อ่านคน" เป็นขั้นเล็กน้อย — ไม่เสียเงินเช่าเครื่องเลย

นี่คือทางแก้ที่ตรงกับสาเหตุจริงที่สุด (คำตอบยาวเกินไป ไม่ใช่ภาพเบลอ) ลองแบ่งให้โมเดลตอบทีละโชนกริตแทนทั้งแผ่น น่าจะยกคะแนน plan_beam ขึ้นได้มากโดยไม่ต้องเปลี่ยนโมเดลเลย

2 เปิดสอน "ตา" ของโมเดลด้วย — ฟรี ไม่เสียเวลาเพิ่ม

ตอนนี้ Constaintant ฝึกสอนแค่ "สมอง" ของ Qwen แต่ปิดการฝึก "ตา" ไว้ (เดิมปิดไว้เพื่อให้ export ไฟล์แบบพิเศษได้ ซึ่งตอนนี้ไม่จำเป็นแล้ว) เปิดฝึกทั้งสองส่วนพร้อมกัน น่าจะช่วยให้อ่านแบบก่อสร้างไทยแม่นยำขึ้น ใช้พื้นที่ความจำเพิ่มน้อยมาก (ต่ำกว่า 1%) ลองได้เลยบนเครื่องเดิม

3 ถ้าจะลองโมเดลใหม่ — ทดสอบอ่านภาพไทย 1 ครั้งก่อน ไม่ต้องทุน

ทั้ง GLM-4.6V และ Llama-4-Scout ไม่มีใครพิสูจน์มาก่อนว่าอ่านภาษาไทยออก ลองถามแบบไม่ทุน (zero-shot) ก่อน 1 ครั้ง ราคาไม่ก็สิบบาท ถ้าอ่านไทยไม่ออกเลยตั้งแต่ต้น ก็ไม่ต้องเสียเงินทุนเต็มรอบ

4 เข้าการตรวจสอบสั้นๆ ก่อนเทรนเต็มรอบเสมอ

กันเสียเงินเทรนเต็มรอบ (หลายชั่วโมง) แล้วมาเจอว่าตั้งค่าผิดพลาด — รันสั้นๆ 5 ก้าวก่อนเสมอ (มีคำสั่ง TEST_STEPS=5 อยู่แล้วในสคริปต์)

5 เทียบผลกับโมเดลเดิมก่อนเปลี่ยนจริง

Qwen3.6-35B-A3B ที่ใช้อยู่ตอนนี้ทำคะแนนอ่านตัวหนังสือดีที่สุดในกลุ่มโมเดลเปิดทั้งหมดที่มีการวัดจริง ต้องแน่ใจว่าตัวใหม่คุ้มค่าที่จะเปลี่ยนจริงๆ ก่อนลงทุนเต็มที

ตัวเลขทั้งหมดตรวจสอบจาก HuggingFace API, GitHub issue tracker, และ vast.ai ณ วันที่ 26 ส.ค. 2569 — ราคาการ์ดจอเข้าเปลี่ยนได้ทุกวัน ควรเช็คซ้ำก่อนกดเช่าจริง ภาพเปรียบเทียบในส่วนที่ 2 ดัดจากไฟล์จริงในชุด

ข้อมูลฝึกสอน (บ้าน_ใหญ่_1ชั้น_01 หน้า15)